Hacking the Data Science Job Description

Metis Data Science Bootcamp, Project 5, March 2021

Garreth Cline

Slides remade in October 2026 with AI help. Every result comes from the original 2021 notebook.

Code and notebook: github.com/garrethcline/Hacking-the-Data-Science-Job-Description

The goal

Use job postings to see which skills data jobs ask for, and how the job titles differ.

- Help job seekers keep up with what employers ask for.
- Help them understand the differences between job titles.
- Supervised part: predict a posting's job title from its description, then look at the words the model relies on for each title.
- Unsupervised part: topic modeling and clustering, to see what groups the postings fall into.

Tools

Python, pandas and NLTK for cleaning

scikit-learn: TF-IDF, LinearSVC, grid search

NMF topic model, MiniBatchKMeans

PCA and t-SNE for the cluster plots

The data

Data job postings scraped from Indeed.com in seven batches

39,209 postings → **11,817** different titles → **150** titles kept as labels → **19,691** with a top-150 title

Cleaning the titles

- Lowercased, removed punctuation, reduced words to their base form
- Removed filler words such as senior, lead, remote and state names, then joined the rest into one label

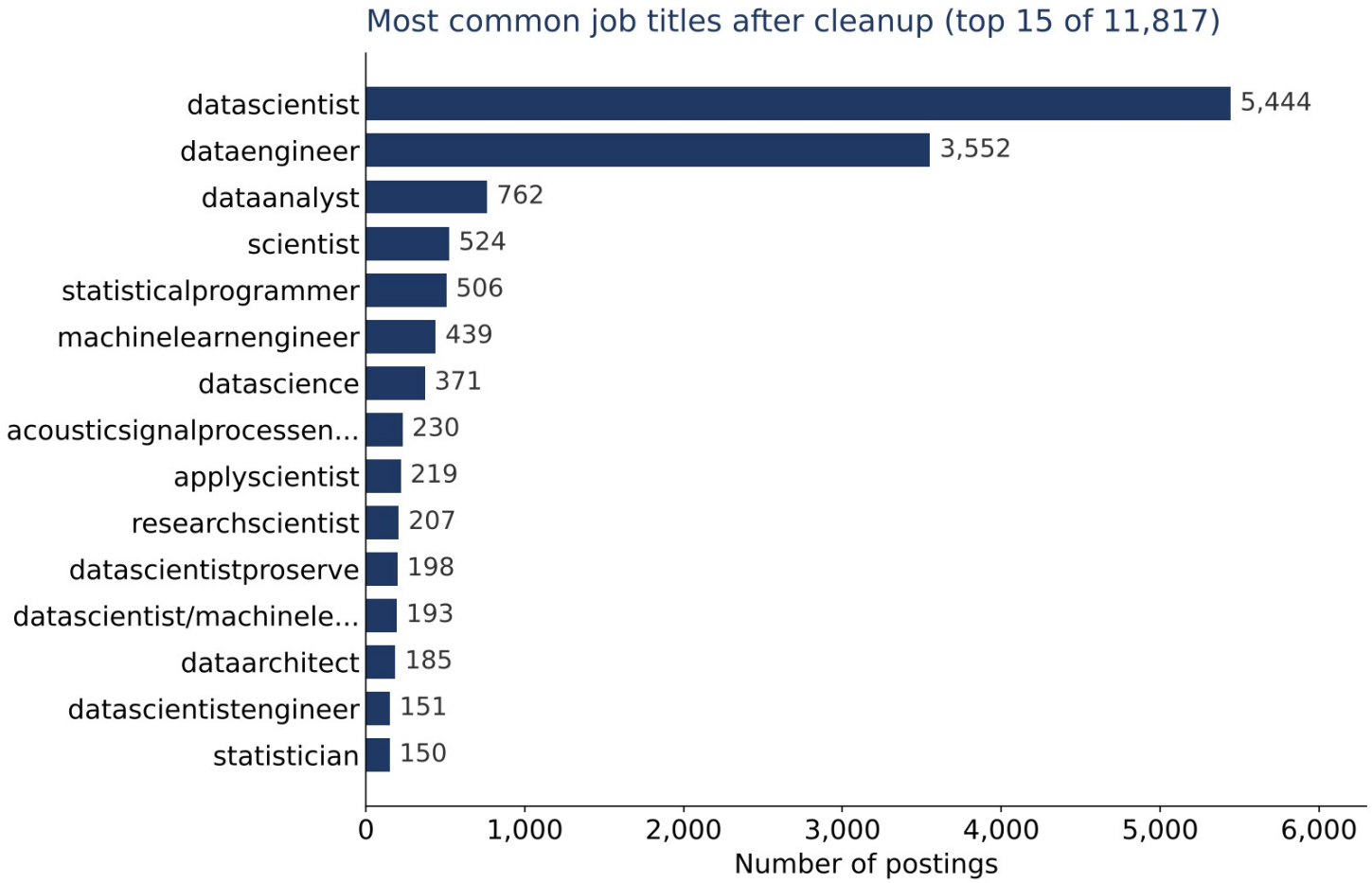
Senior Data Scientist, Remote → datascientist

Machine Learning Engineer → machinelearnengineer

Cleaning the descriptions

- Same cleanup, plus removing title words (data, scientist, engineer and the 150 labels) so the model couldn't read the title back out of the text
- Turned each description into TF-IDF word scores
- Held out 20% (3,939 postings) as a test set

The most common job titles



Data scientist and data engineer make up about 23% of the 39,209 postings.

After them the counts drop fast. There are 11,817 different titles, and most appear only a few times.

Some titles come from one employer posting the same job many times, like the acoustic signal processing title with 230 postings.

Counts from the 2021 notebook.

Predicting the job title

One LinearSVC yes or no classifier per title (one-vs-rest), trained on the descriptions of the 19,691 postings.

A grid search tried C = 1, 10, 100 and 1,000 with 30-fold cross-validation: 120 fits, 14 minutes. The best value was C = 100.

I started a 12-model comparison like the one in Project 3 but stopped it before it finished, so there is no baseline model to compare against.

88.3

Jaccard score

Out of 100; higher is better

0.12%

Hamming loss

Share of yes or no calls that were wrong

92.9%

Precision

Share of predicted titles that were right

88.9%

Recall

Share of test postings whose title was found

Test set: 3,939 postings. Precision and recall were added in 2026, from the notebook's per-title confusion matrices.

Results by title

Title (as cleaned)	Test postings	Recall	Precision
datascientist	1,106	93%	92%
dataengineer	697	91%	89%
dataanalyst	168	72%	83%
scientist	125	92%	97%
statisticalprogrammer	97	100%	99%
datascience	90	66%	86%
machinelearnengineer	74	66%	80%
applyscientist	39	77%	88%
researchscientist	36	67%	92%

94 of the 150 titles were predicted perfectly. Many of them are titles used by a single employer, whose postings repeat the same text and company name.

Copies of those postings were probably in both the training and test sets, so the overall score is likely higher than it would be on new postings.

The common titles are a fairer test: data scientist 93% recall, data analyst 72%, machine learning engineer 66%.

What the model learned

The ten words with the most weight for each title in the final model, strongest first

Title	Words
datascientist	unlimited, ser, statistics, python, predictive, nine, div, link, vastly, algorithms
dataengineer	et, pipelines, warehouse, exposure, storage, pipeline, integrity, neptune, eustis, kafka
dataanalyst	analysis, uss, rf, moca, query, sql, requirements, neil, circulate, analyse
machinelearnengineer	production, model, ml, algorithms, ker, tensor, train, memory, eli, zing
statisticalprogrammer	output, tfl, trial, affairs, adam, table, datasets, macro, list, multiple
businessintelligenceanalyst	pareto, wheel, flipside, strategic, bi, report, greenville, visualization, tableau, conga

Some are real skills: sql and query for analysts, pipelines, warehouse and kafka for data engineers, tableau for BI analysts, and clinical trial reporting terms (tfl, adam, macro) for statistical programmers. Many are noise the cleanup missed, such as word fragments (ser, et, ker) and place names (eustis, greenville).

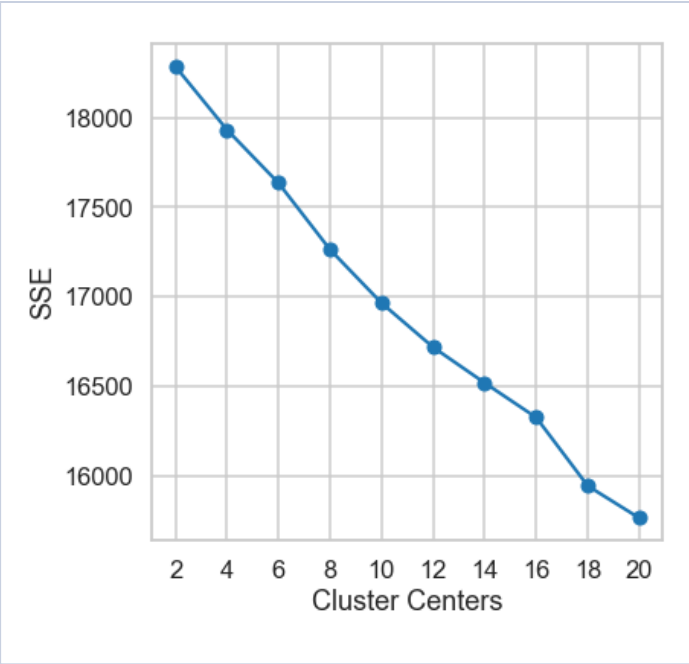
Topic model

18 topics from NMF on the 19,691 descriptions. I added the labels in 2026; the words are from the 2021 notebook.

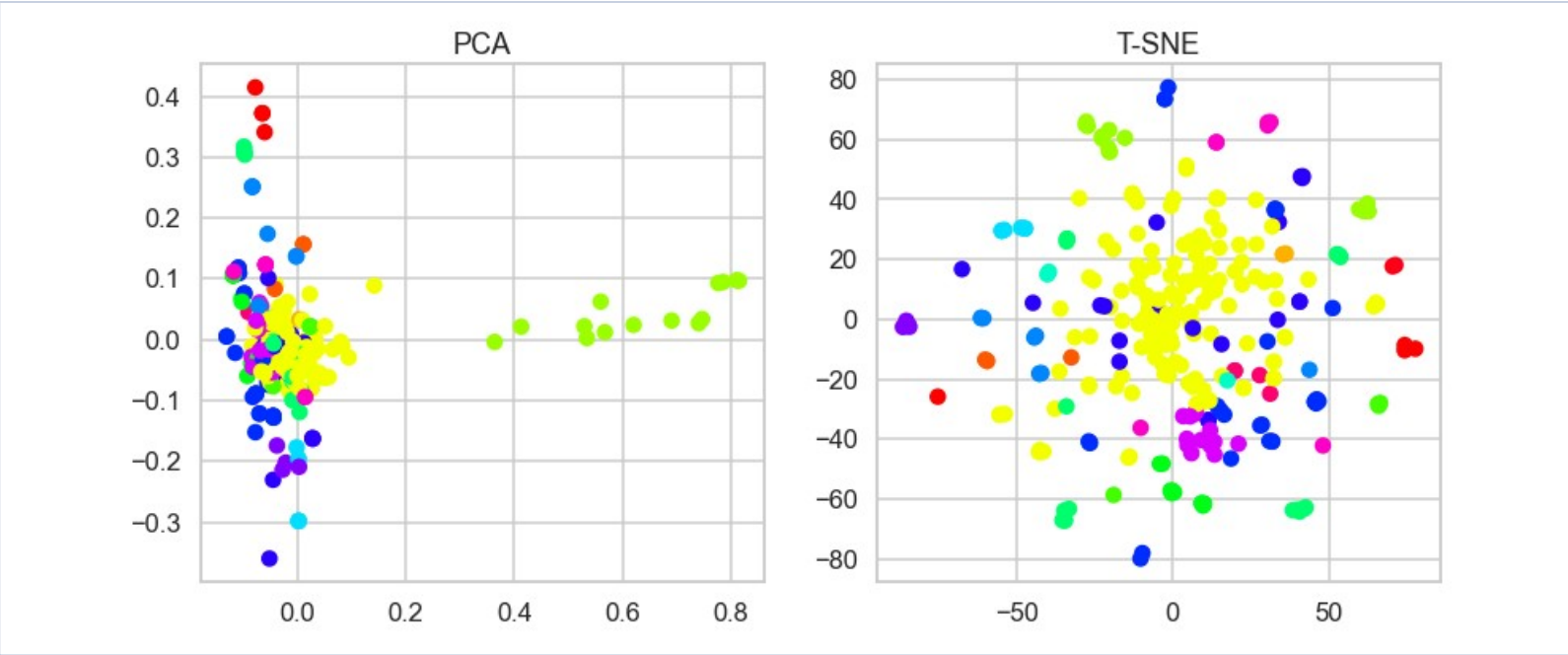
Label	Top words in the topic
Amazon	amazon, ml, aw, web, model, customers, professional, inc
Intuit	intuit, mountain, view, prosperity, model, products, side, quick
Facebook	facebook, closer, together, people, accommodations, build, ways, menlo
Acoustic signal processing	signal, ssc, corporation, underwater, acoustic, acoustics, systems, sonar
Government intelligence	dni, aug, analytic, tyson, corner, advise, efforts, clearance
Drug studies	study, td, drug, project, biology, university, unite, include
Agriculture	ag, nov, growers, map, agriculture, crop, labor, spatial
General data work	team, cloud, build, design, review, model, financial, tool

Several topics are one employer's postings. Others group by field, such as drug studies and agriculture.

Clustering



Error by number of clusters



300 random postings, colored by cluster

MiniBatchKMeans on the same descriptions. The error kept falling as clusters were added, with no clear bend, so I used 18, the same as the topic model. Most postings fall in one large group in the plots.

The clusters looked like the topics: single employers (Amazon, Intuit, Facebook) and fields such as university research, clinical trials and agriculture. Charts from the 2021 notebook.

What I would do differently

- Remove duplicate postings before splitting the data, so the test set only has postings the model hasn't seen.
- Strip company names, locations and benefits text from the descriptions.
- Group the 11,817 titles into a handful of job families before modeling.
- Finish the model comparison, with a dummy baseline.
- Rerun the notebook from top to bottom. It was run in several sessions, so some printed outputs come from earlier runs than the final model.
- Scrape new postings; these are from 2021.